Lab 3: Reinforcement Learning

Manfred Clase (mancl774)

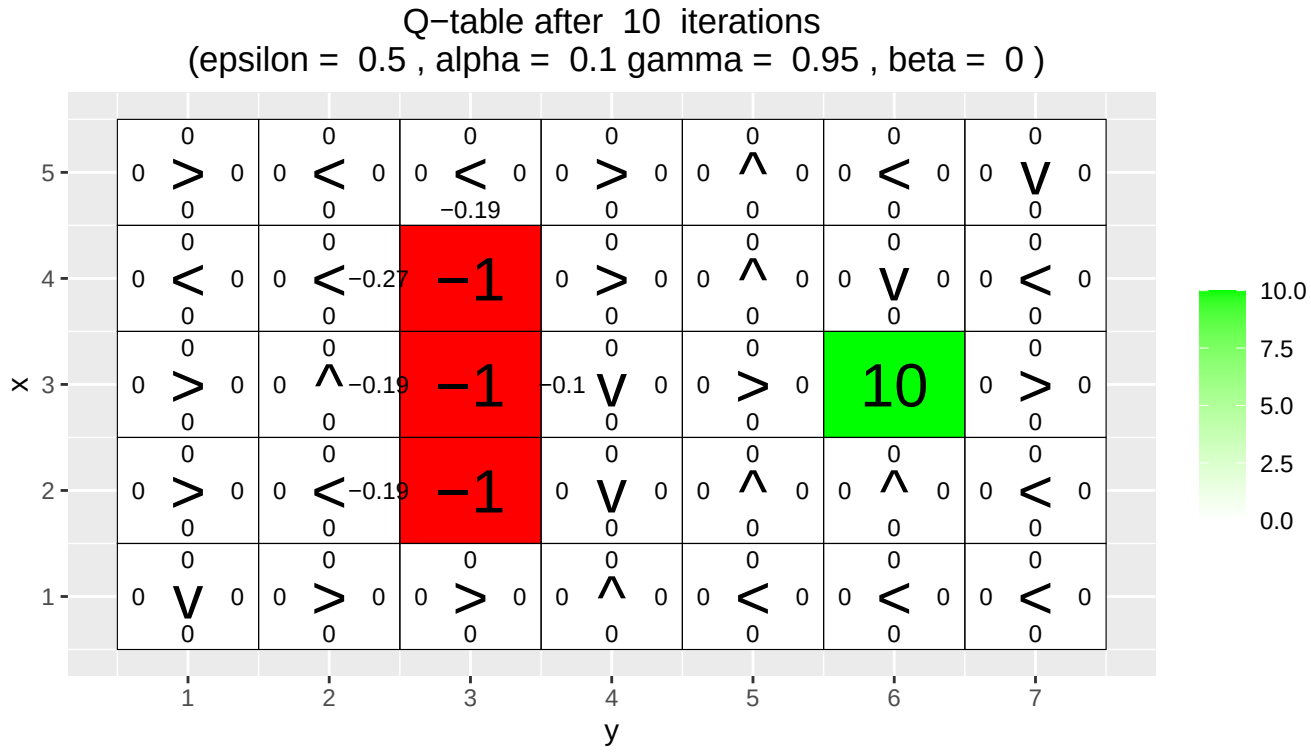
2026-10-05

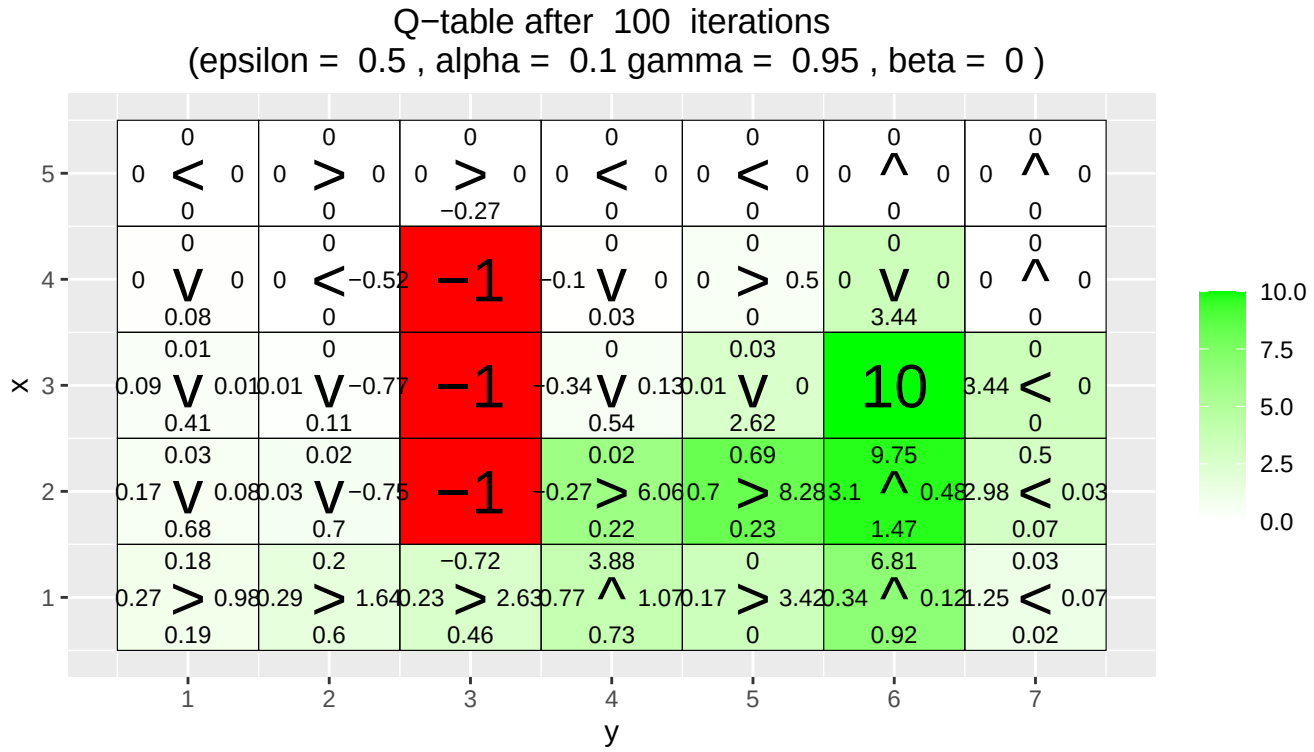
2.1. Q-learning

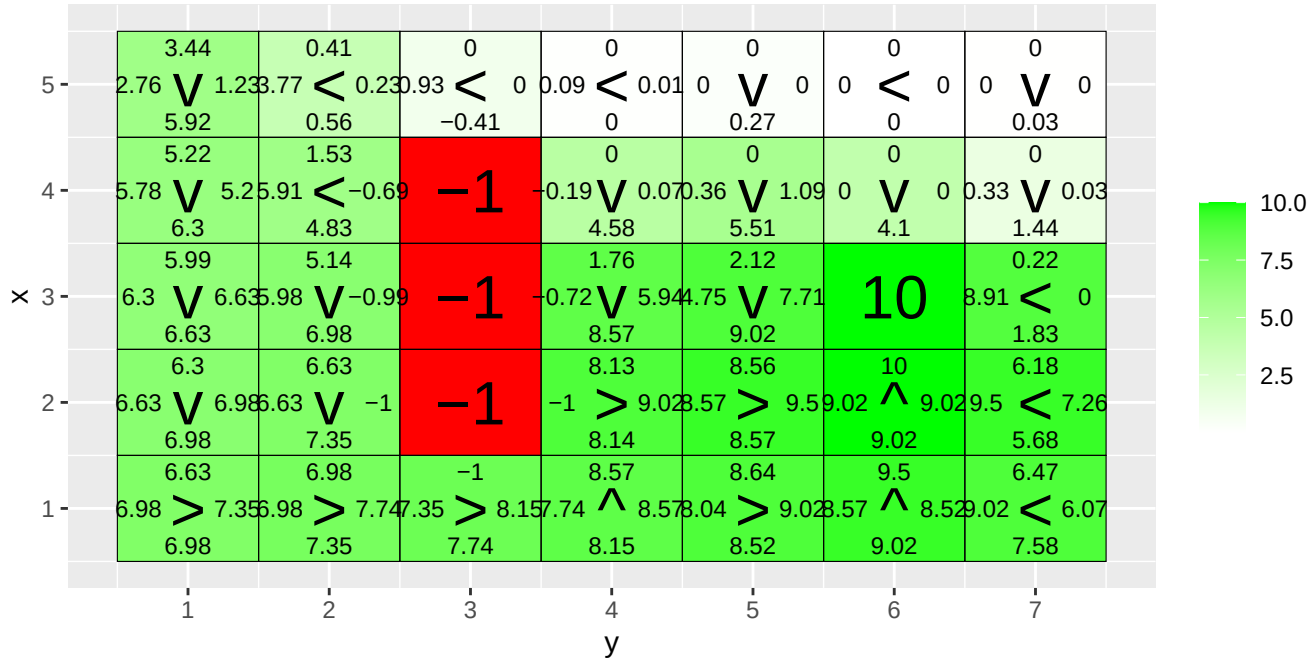
The Q-learning code (epsilon-greedy policy, transition model and the temporal-difference update) is listed in the appendix.

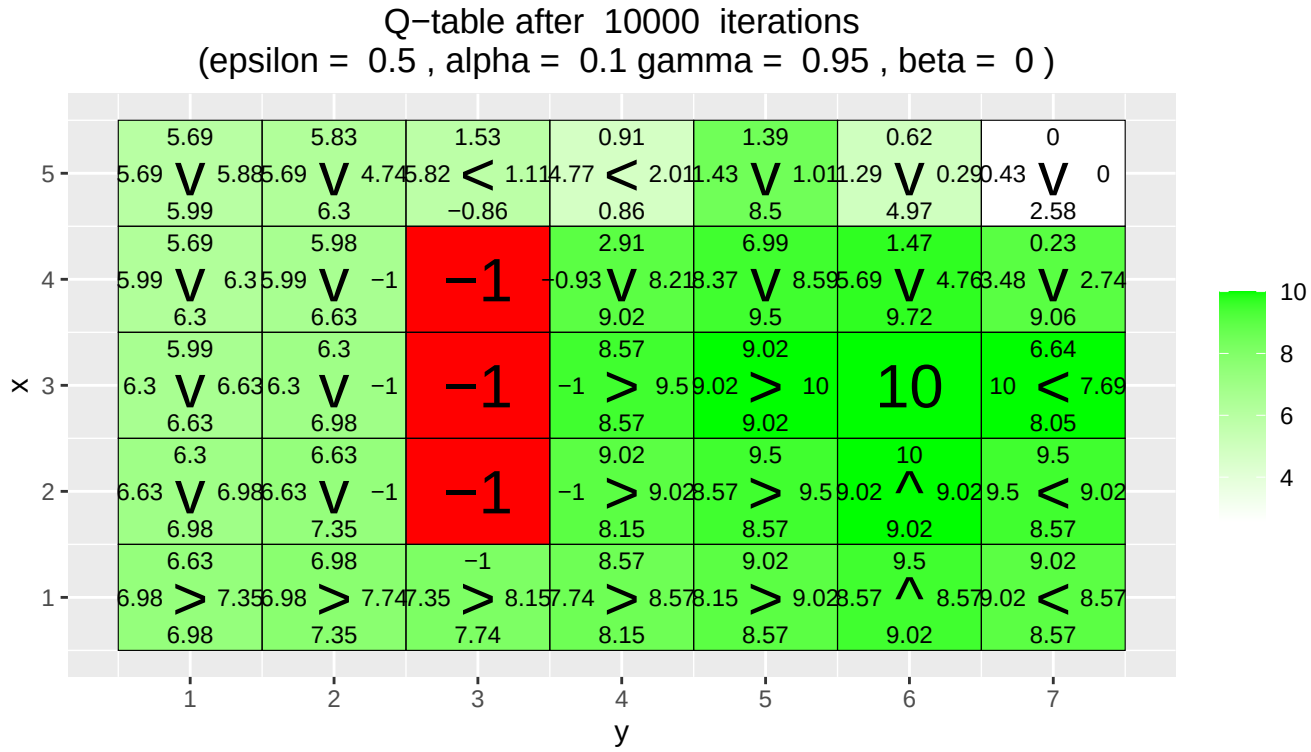
2.2 Environment A

The agent starts in (3,1), the goal (+10) is at (3,6) and the states (2,3), (3,3) and (4,3) give -1. The Q-table after 10, 100, 1000 and 10000 episodes is shown below.




Q−table after 100 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
x
y
10.0
7.5
5.0
2.5
0.0


Q−table after 1000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
x
y
10.0
7.5
5.0
2.5



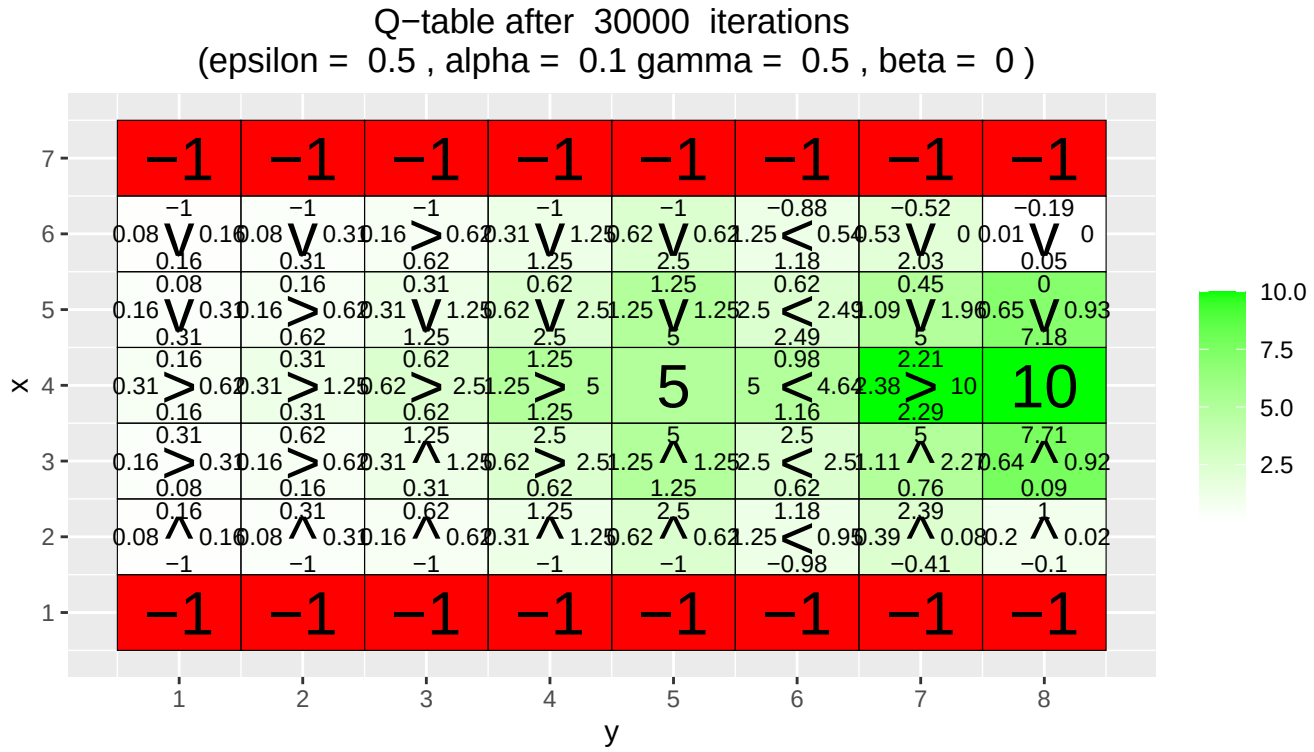
What has the agent learned after the first 10 episodes? Almost nothing. Most Q-values are still 0. The few that changed are mainly small negative values for moves into the -1 states near the start, plus a small positive value next to the goal if it was reached.

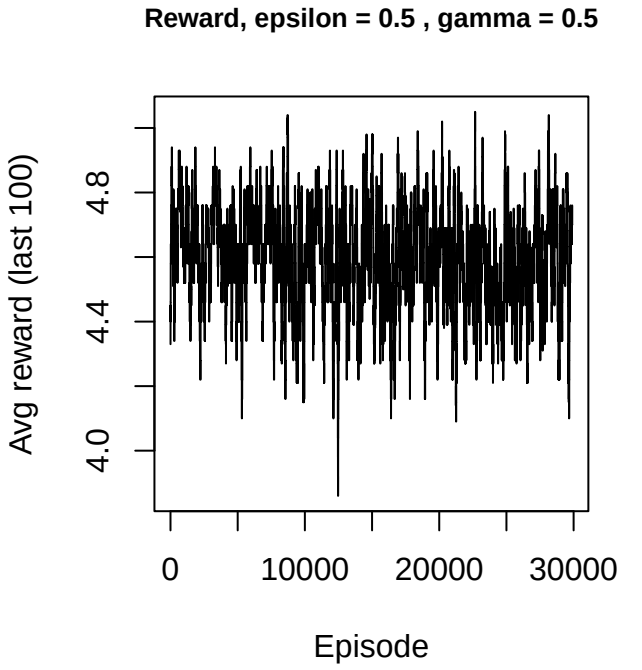
Is the final greedy policy optimal for all states? No. States on the common paths from the start are visited often and get the right action, but states that are rarely visited get few updates, so their greedy action can be wrong. Convergence requires that every state-action pair is visited often enough, which does not happen here since the agent always starts in (3,1).

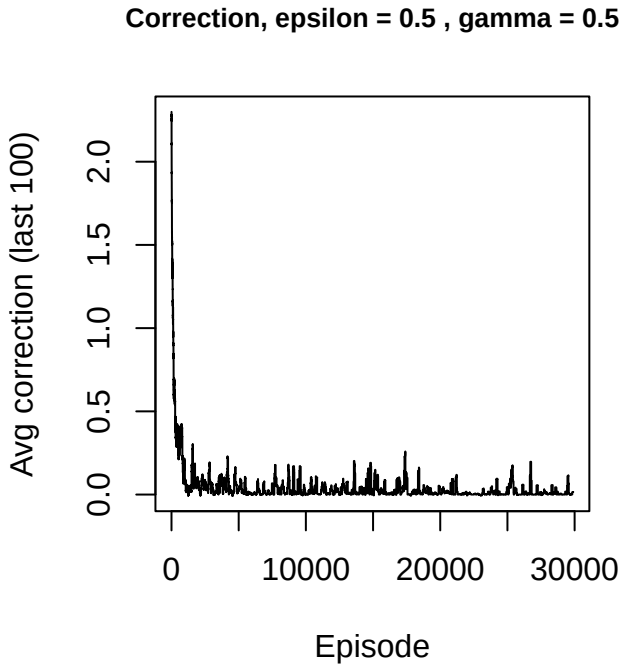
Do the Q-values reflect the multiple paths to the goal? Not really. The Q-table mostly reflects one route (above or below the -1 states). Once one route looks slightly better the agent uses it more, so it gets updated more and the other route lags behind. More exploration (higher epsilon, more episodes or random start states) would make both paths show up.

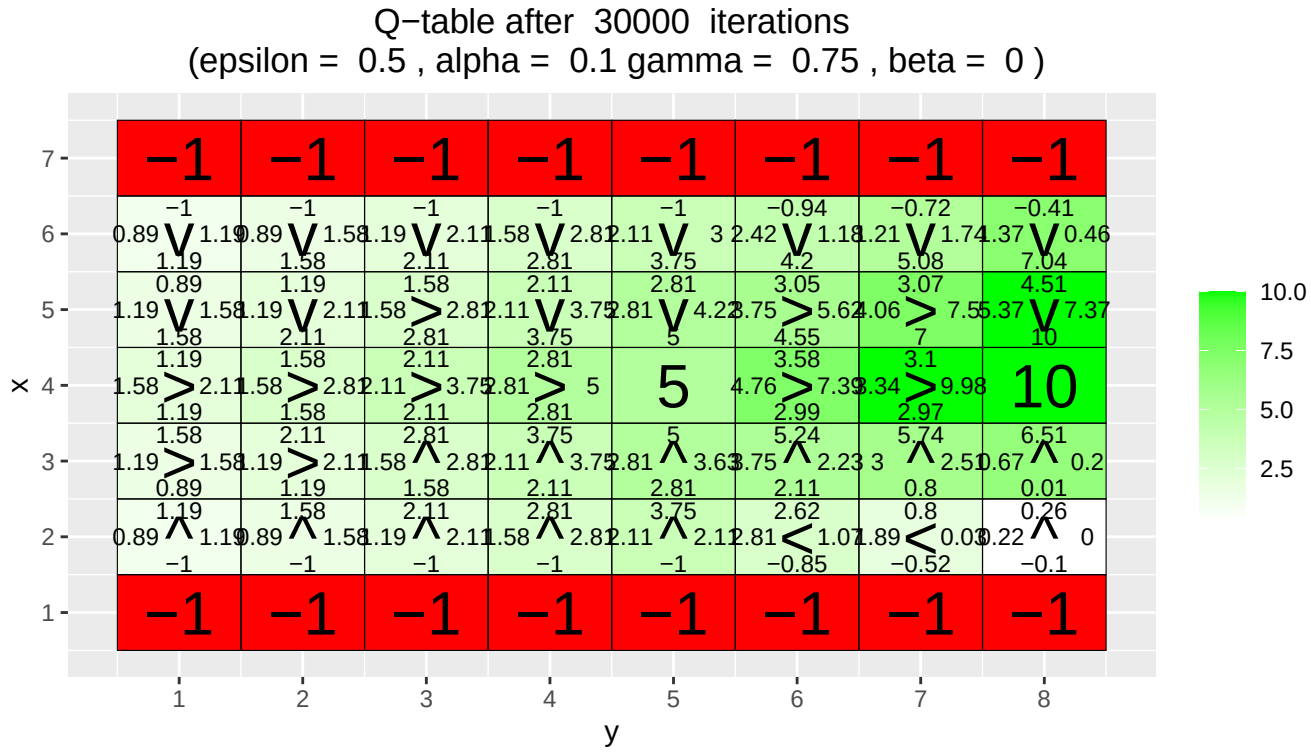
2.3 Environment B: effect of epsilon and gamma

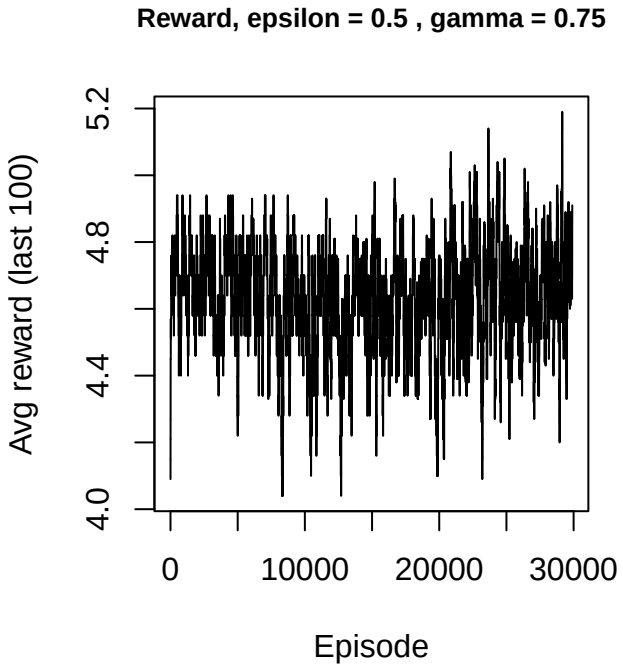
The top and bottom rows give -1, there is a +5 at (4,5) and a +10 at (4,8). The agent starts in (4,1). For each setting the Q-table is shown together with the moving average of the reward and of the correction.

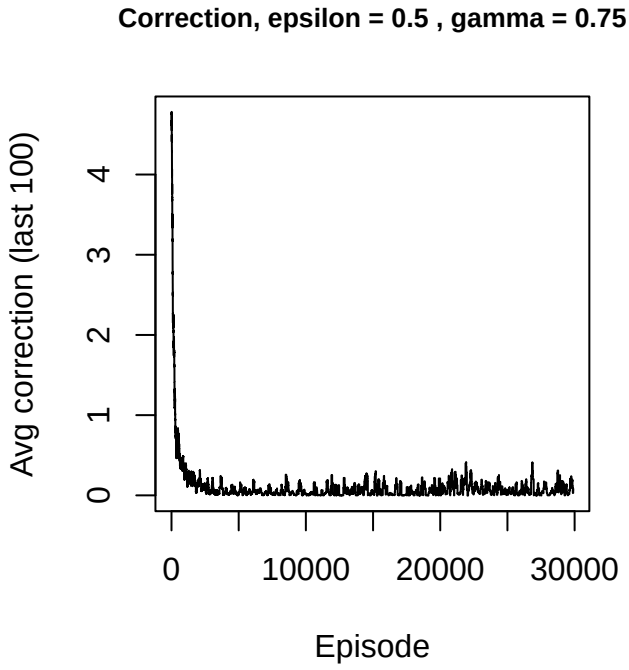

Q−table after 30000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.5 , beta = 0)
x
y
10.0
7.5
5.0
2.5

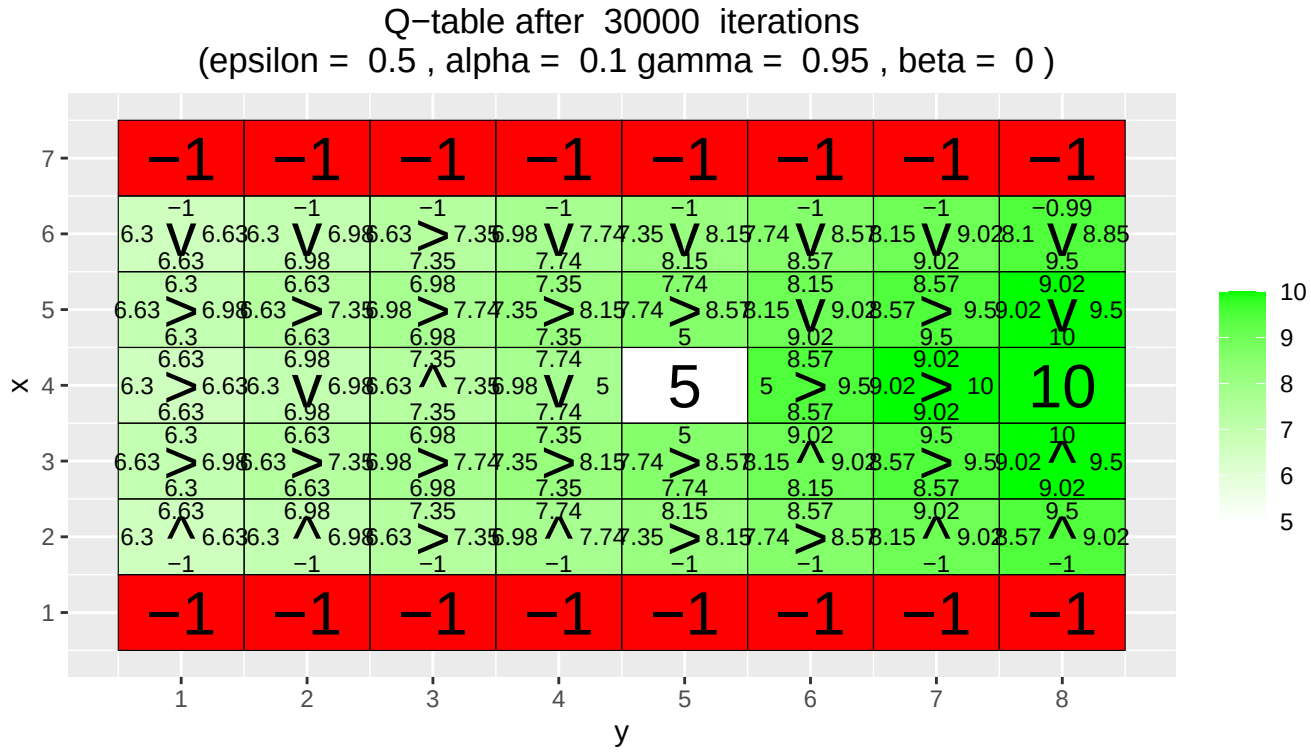

Reward, epsilon = 0.5 , gamma = 0.5
Avg reward (last 100)
Episode

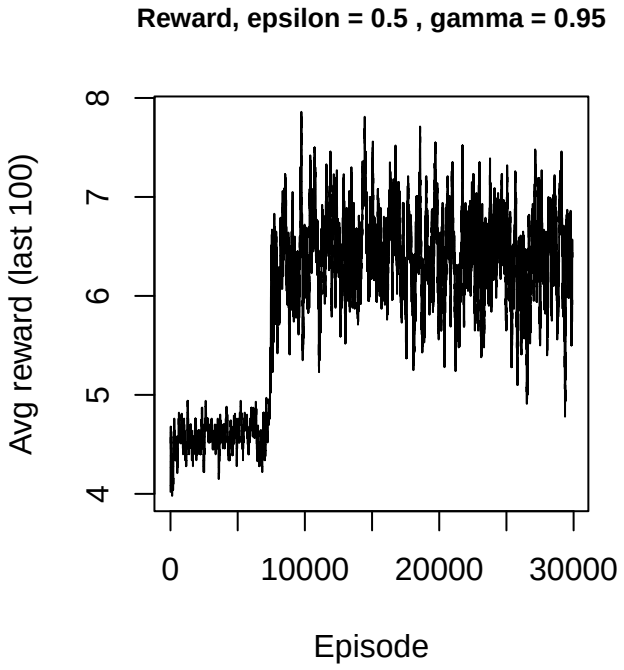

Correction, epsilon = 0.5 , gamma = 0.5
Avg correction (last 100)
Episode

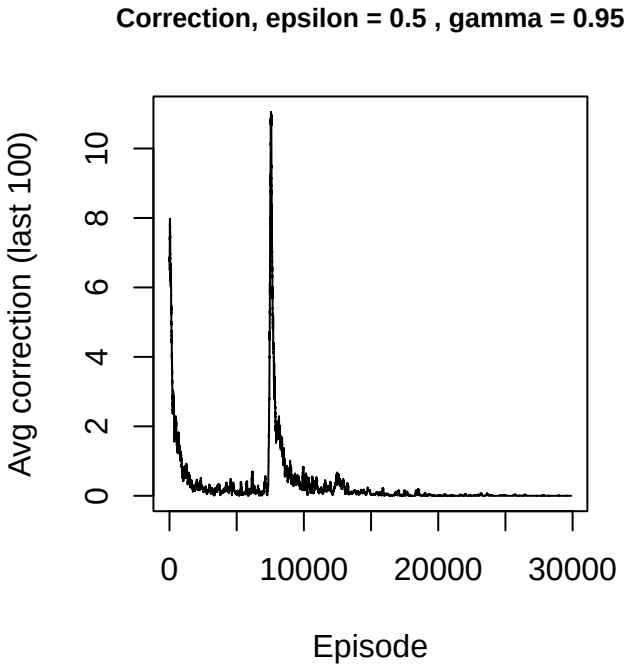

Q−table after 30000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.75 , beta = 0)
x
y
10.0
7.5
5.0
2.5

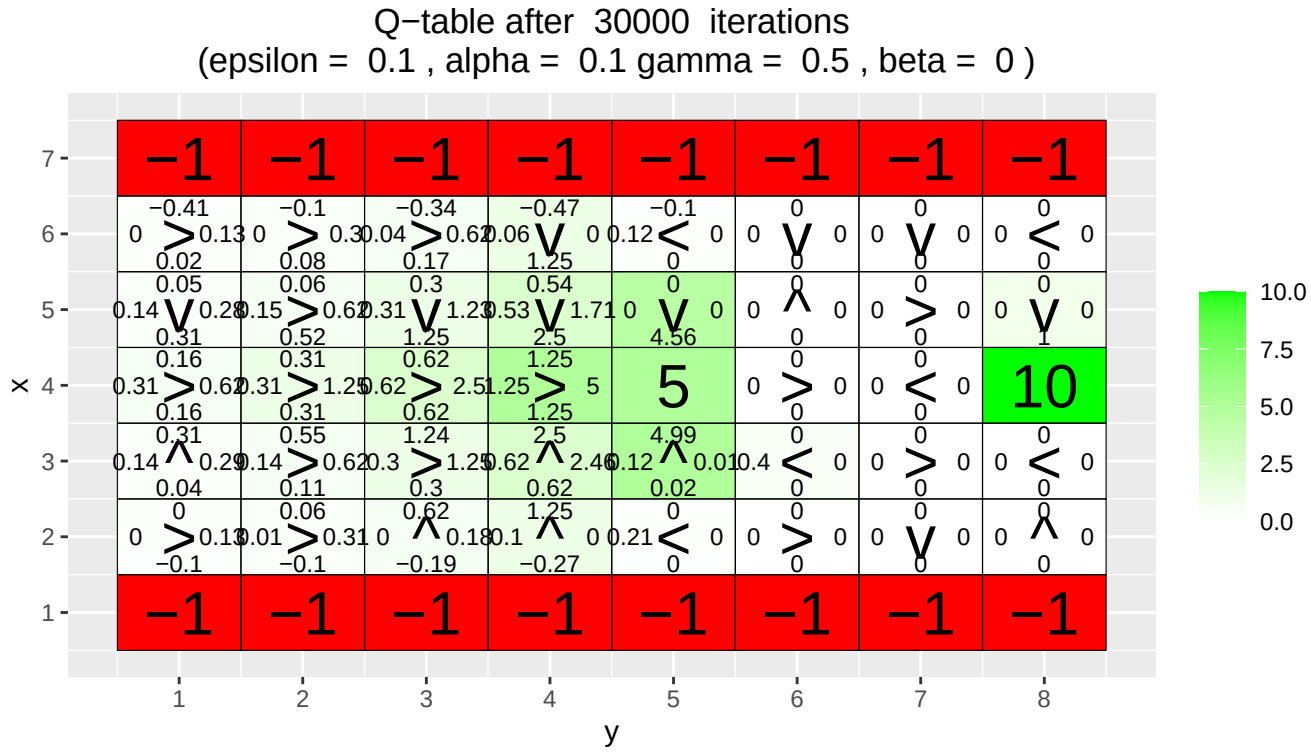

Reward, epsilon = 0.5 , gamma = 0.75
Avg reward (last 100)
Episode

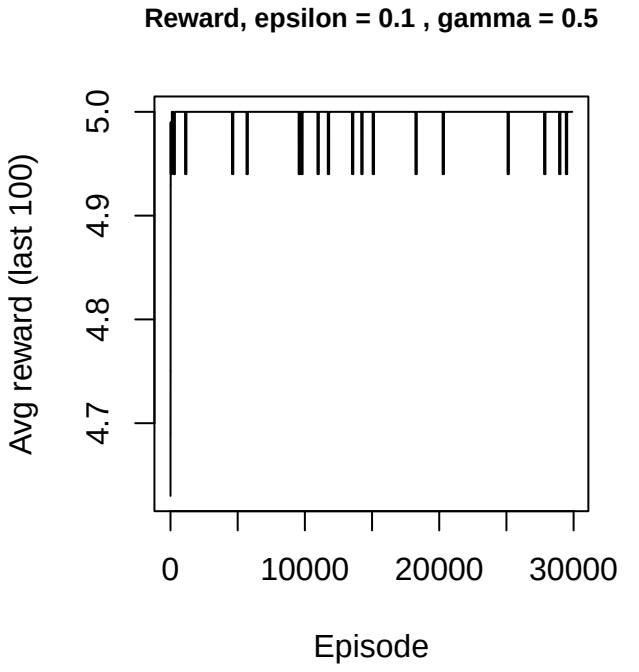

Correction, epsilon = 0.5 , gamma = 0.75
Avg correction (last 100)
Episode

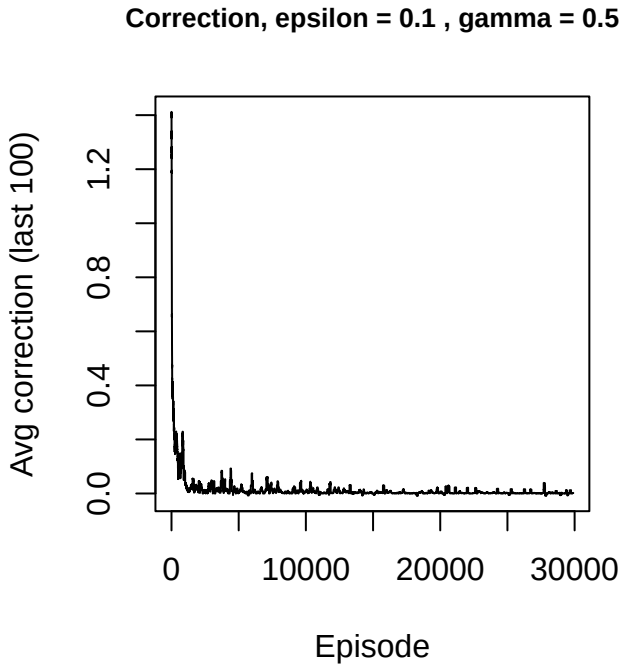

Q−table after 30000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
x
y
10
9
8
7
6
5

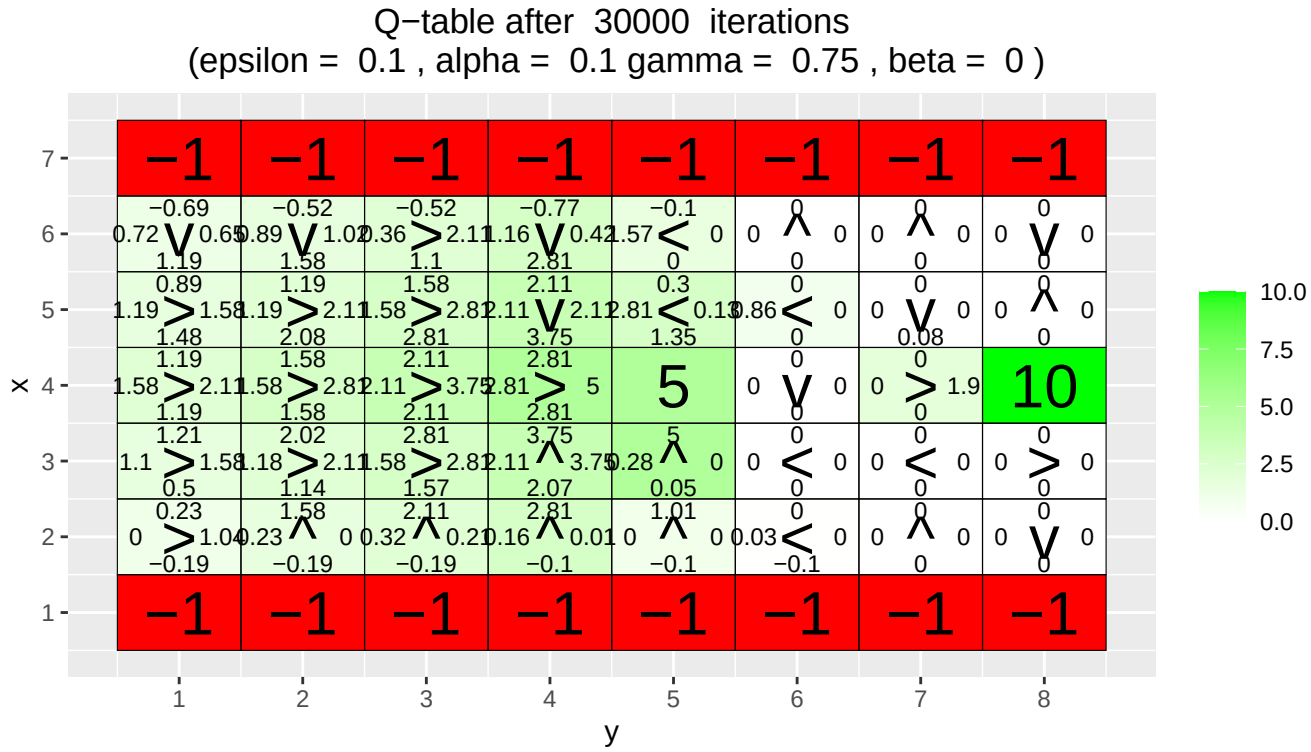

Reward, epsilon = 0.5 , gamma = 0.95
Avg reward (last 100)
Episode

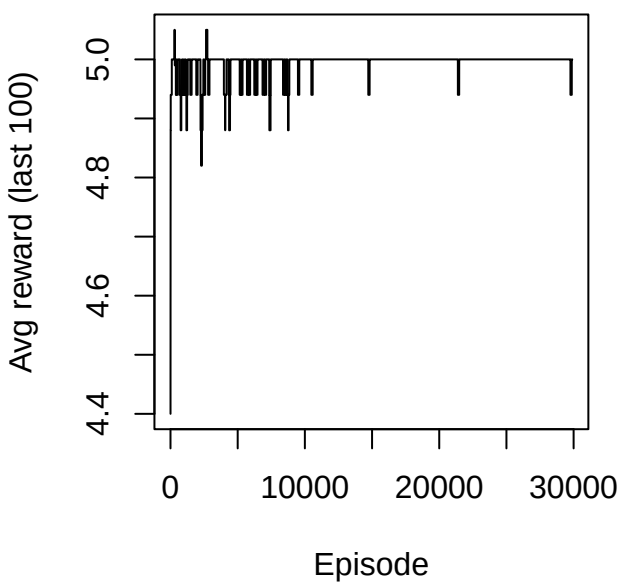

Correction, epsilon = 0.5 , gamma = 0.95
Avg correction (last 100)
Episode

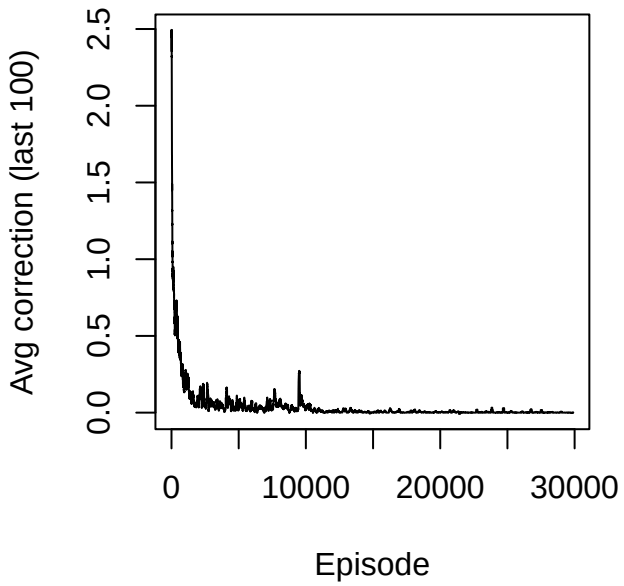

Q−table after 30000 iterations
(epsilon = 0.1 , alpha = 0.1 gamma = 0.5 , beta = 0)
−1
5
10
x
y
10.0
7.5
5.0
2.5
0.0

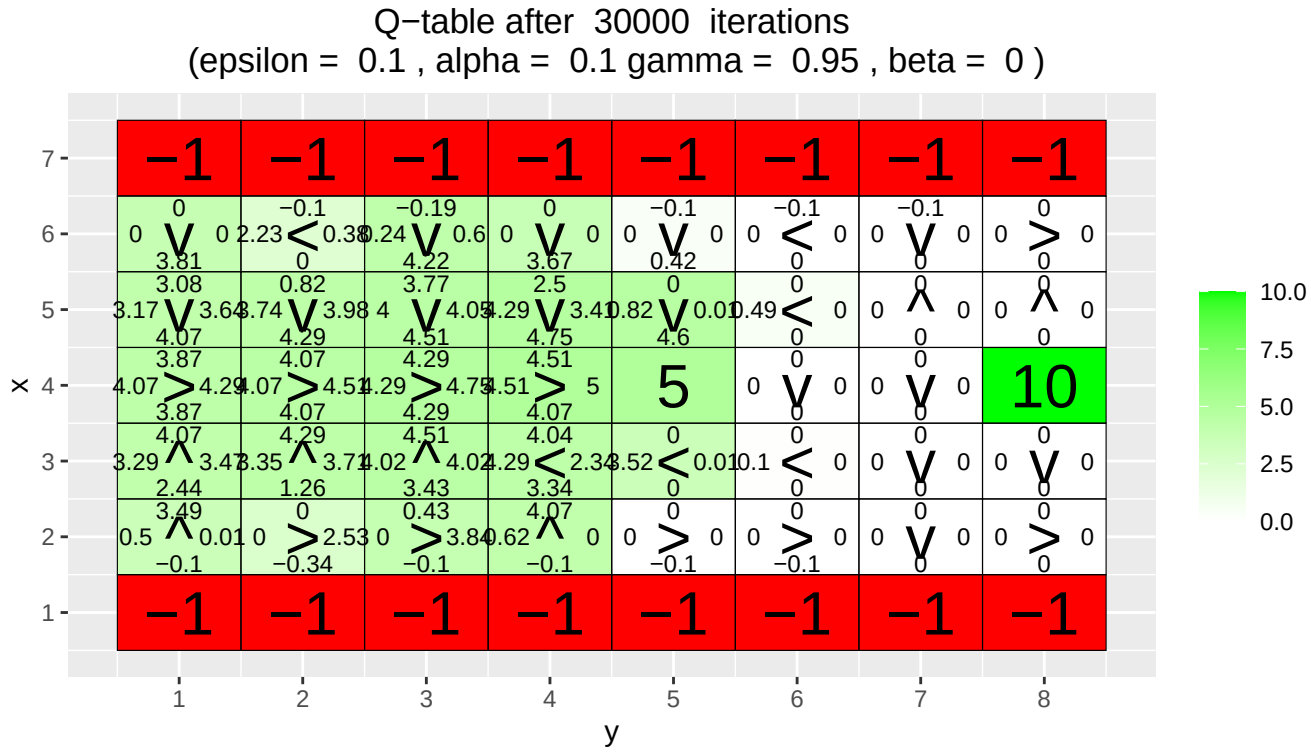

Reward, epsilon = 0.1 , gamma = 0.5
Avg reward (last 100)
Episode

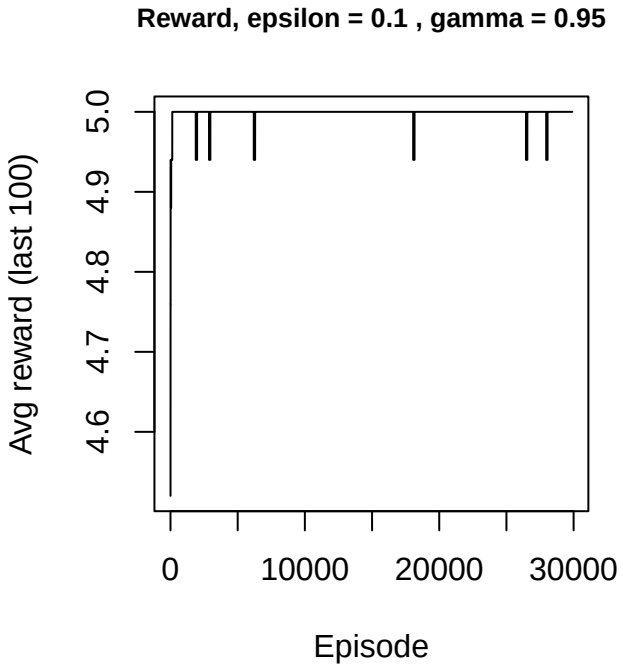

Correction, epsilon = 0.1 , gamma = 0.5
Avg correction (last 100)
Episode

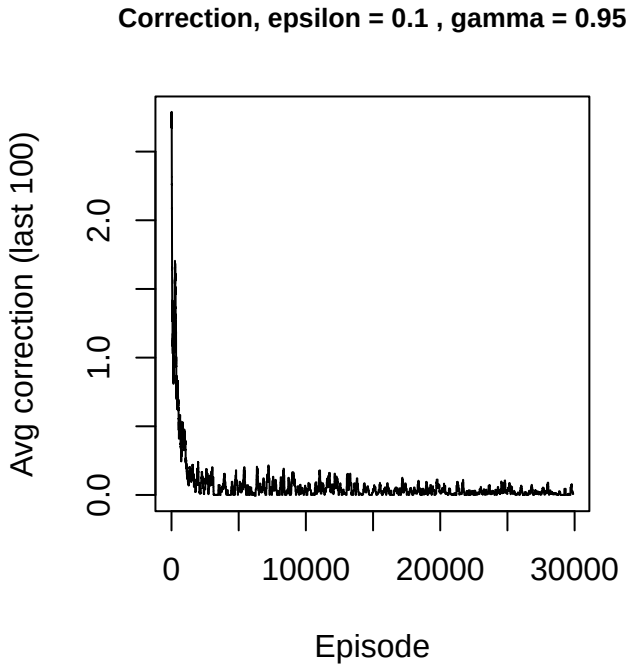

Q−table after 30000 iterations
(epsilon = 0.1 , alpha = 0.1 gamma = 0.75 , beta = 0)
x
y
10.0
7.5
5.0
2.5
0.0


Reward, epsilon = 0.1 , gamma = 0.75
Avg reward (last 100)
Episode


Correction, epsilon = 0.1 , gamma = 0.75
Avg correction (last 100)
Episode







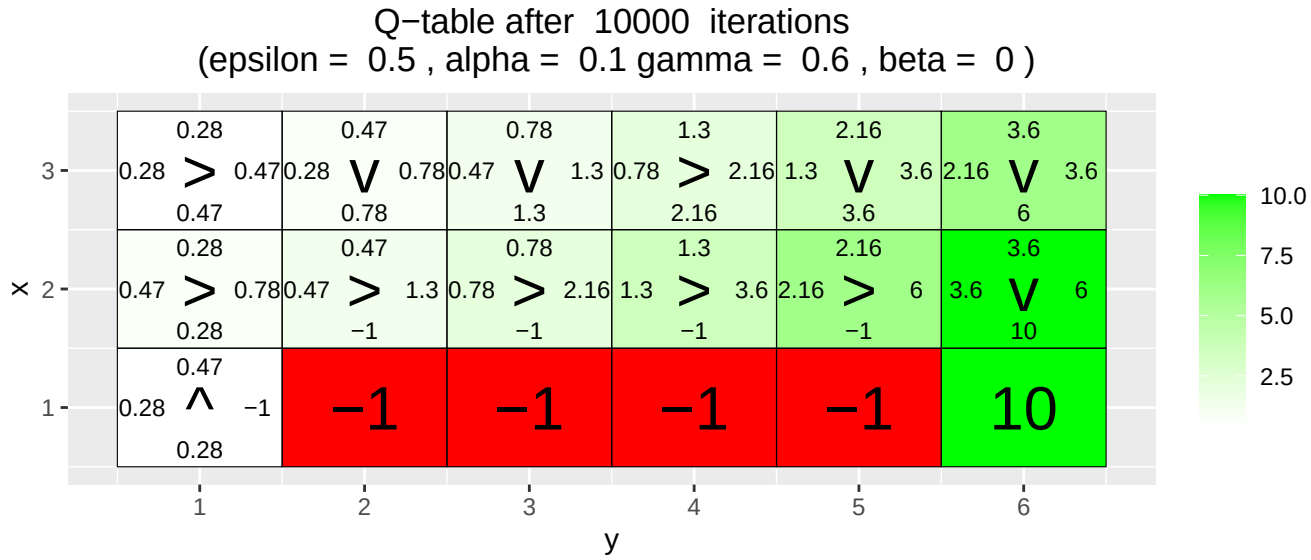
Observations. With gamma = 0.5 and 0.75 the agent goes for the closer 5, since the discounted value of the farther 10 is lower. From the start the 5 is 4 steps away and the 10 about 9 steps (the agent has to walk

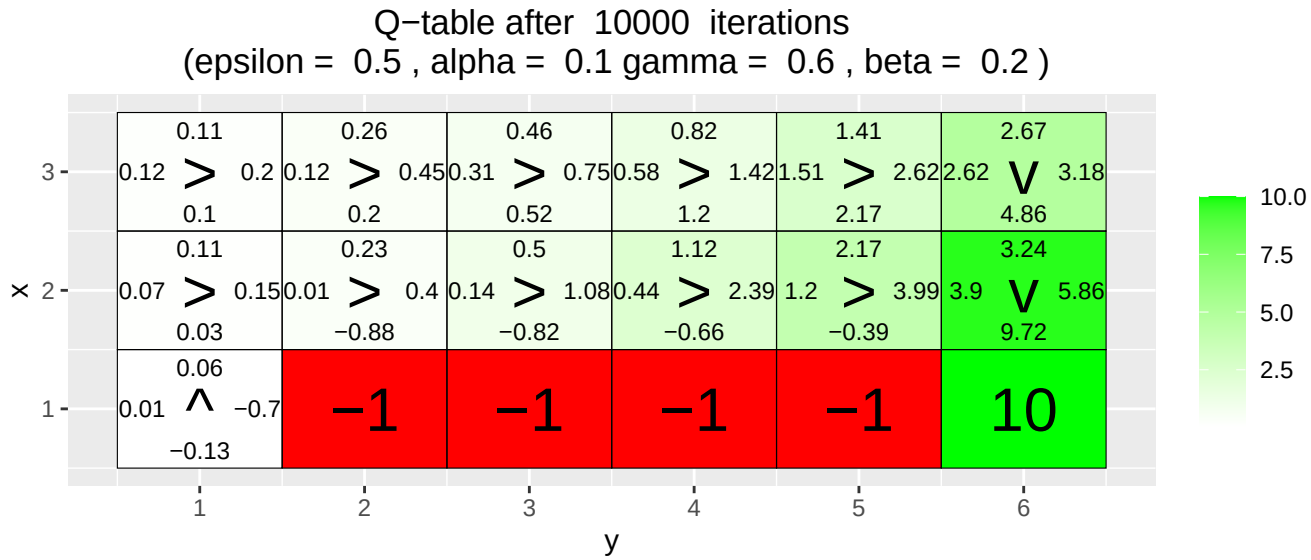
around the 5), so the values are roughly $5 \cdot \text{gamma}^3$ vs $10 \cdot \text{gamma}^8$: 0.63 vs 0.04 for gamma = 0.5, 2.11 vs 1.0 for gamma = 0.75, and 4.29 vs 6.63 for gamma = 0.95. So with gamma = 0.95 the 10 is worth the detour and the agent goes around the 5 to reach it. The Q-values along the path also shrink by a factor gamma per step away from the reward.

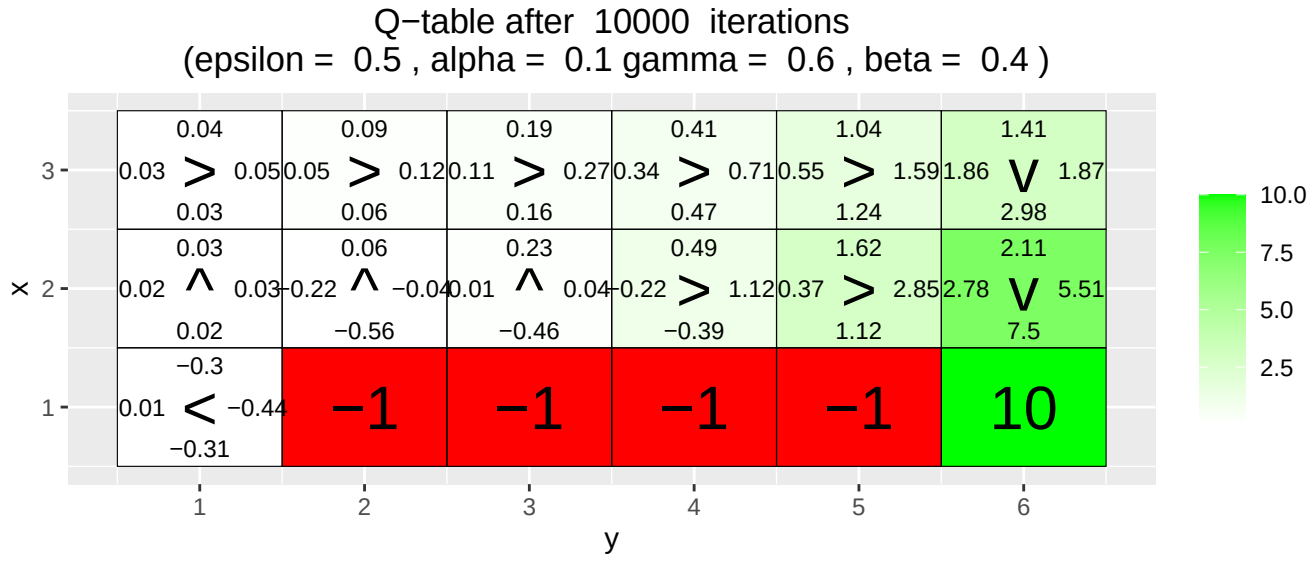
With epsilon = 0.1 the agent explores less. It gets a higher average reward, since fewer random moves end in the -1 rows, but large parts of the grid stay unexplored (Q-values remain 0) and only one path is learned. Finding the 10 requires getting around the 5, which is harder with little exploration. In all settings the correction drops towards 0, meaning the Q-table has converged.

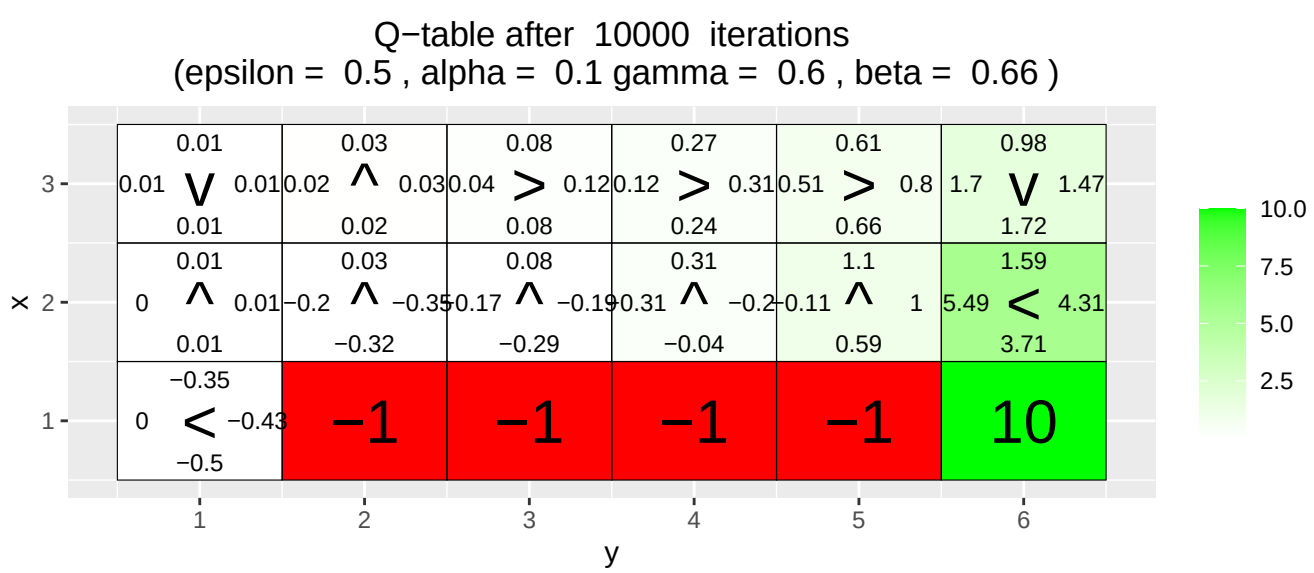
2.4 Environment C: effect of beta

The agent starts in (1,1), the states (1,2)-(1,5) give -1 and the goal (+10) is at (1,6). We use epsilon = 0.5 and gamma = 0.6 and vary beta.









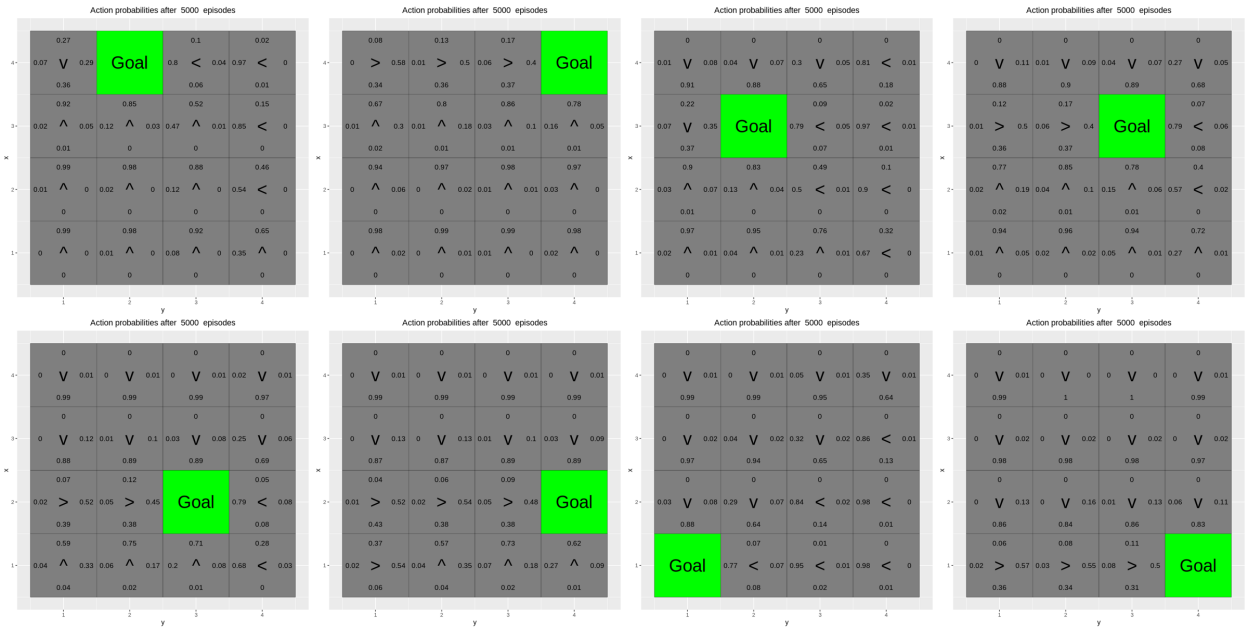
Observations. With beta = 0 there is no slipping, so the agent takes the shortest path along row 2, right next to the -1 states, and the Q-values decay by gamma = 0.6 per step (10, 6, 3.6, 2.16, ...). With beta = 0.2 the route is mostly the same, but the Q-values are lower and moves next to the -1 states get negative values, since slipping into them is now possible. With beta = 0.4 and 0.66 the agent avoids row 2 and moves up to row 3, taking a longer but safer path. In the start state it can even choose "left" into the wall: it then only moves if it slips, and the slips go up (safe) or down (wall), so it can never slip right into the -1. Overall, higher beta gives lower Q-values and a more cautious policy that trades path length for safety.

2.5 REINFORCE

The code and results are in `RL_Lab2_Colab.ipynb`. The policy is a neural network (two hidden layers with 32 units) with input (x, y, goal x, goal y) and output the probability of each of the four actions.

2.6 Environment D

Eight goals are used for training and the other eight for validation. The validation results after 5000 episodes are shown below.

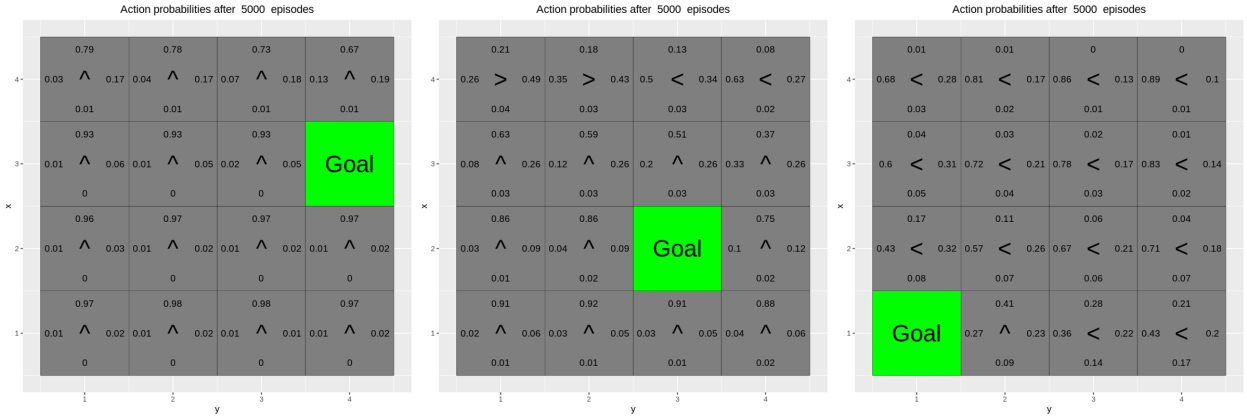


Has the agent learned a good policy? Mostly yes. For the validation goals, which were never used in training, the arrows point toward the goal in almost all states. The probabilities are high far from the goal (around 0.85-0.99) and more spread out near it (around 0.4-0.6). The network has learned to use the goal coordinates, so it generalizes to new goals, since the training goals cover the whole grid.

Could Q-learning have been used? No. Tabular Q-learning learns one fixed goal. Even with the goal added to the state, each (position, goal) pair is a separate table entry, so the entries for the validation goals would never be trained. The neural network can generalize to unseen goals; a table cannot.

2.7 Environment E

All training goals are in the top row, and the validation goals (3,4), (2,3) and (1,1) are in the rows below.



Has the agent learned a good policy? No. Since all training goals are in the top row, the agent mostly learned to go up, plus left/right along the top. It does not use the goal's row. For the validation goals it fails: for goal (3,4) the top row points up into the wall, for goal (2,3) the states below it point up past the goal, and for goal (1,1) most states point left but not down.

Why do the results for D and E differ? In D the training goals cover the whole grid, so the network learns how the goal coordinates relate to the best action. In E the goal's row is always 4, so the network never sees that coordinate vary and cannot learn what it means. It only generalizes to goals similar to the ones it was trained on.

Appendix: R code

```
set.seed(12345)
library(ggplot2)

arrows <- c("^", ">", "v", "<")
action_deltas <- list(c(1,0), # up
                      c(0,1), # right
                      c(-1,0), # down
                      c(0,-1)) # left

vis_environment <- function(iterations=0, epsilon = 0.5, alpha = 0.1, gamma = 0.95, beta = 0){
  df <- expand.grid(x=1:H,y=1:W)
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,1],NA),df$x,df$y)
  df$val1 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,2],NA),df$x,df$y)
  df$val2 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,3],NA),df$x,df$y)
  df$val3 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,4],NA),df$x,df$y)
  df$val4 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y)
    ifelse(reward_map[x,y] == 0,arrows[GreedyPolicy(x,y)],reward_map[x,y]),df$x,df$y)
  df$val5 <- as.vector(foo)
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,max(q_table[x,y,]),
                                     ifelse(reward_map[x,y]<0,NA,reward_map[x,y])),df$x,df$y)
  df$val6 <- as.vector(foo)

  print(ggplot(df,aes(x = y,y = x)) +
          scale_fill_gradient(low = "white", high = "green", na.value = "red", name = "") +
          geom_tile(aes(fill=val6)) +
          geom_text(aes(label = val1),size = 3,nudge_y = .35,na.rm = TRUE) +
          geom_text(aes(label = val2),size = 3,nudge_x = .35,na.rm = TRUE) +
          geom_text(aes(label = val3),size = 3,nudge_y = -.35,na.rm = TRUE) +
          geom_text(aes(label = val4),size = 3,nudge_x = -.35,na.rm = TRUE) +
          geom_text(aes(label = val5),size = 8) +
          geom_tile(fill = 'transparent', colour = 'black') +
          ggtitle(paste("Q-table after ",iterations," iterations\n",
                        "(epsilon = ",epsilon,", alpha = ",alpha,"gamma = ",gamma,", beta = ",beta,")")) +
          theme(plot.title = element_text(hjust = 0.5)) +
          scale_x_continuous(breaks = c(1:W),labels = c(1:W)) +
          scale_y_continuous(breaks = c(1:H),labels = c(1:H)))
```

```
}

GreedyPolicy <- function(x, y){
  # Break ties at random among the actions with maximal Q-value.
  q <- q_table[x,y,]
  best <- which(q == max(q))
  return(best[sample(length(best), 1)])
}

EpsilonGreedyPolicy <- function(x, y, epsilon){
  if(runif(1) < epsilon){
    return(sample(1:4,1))
  } else{
    return(GreedyPolicy(x,y))
  }
}

transition_model <- function(x, y, action, beta){
  delta <- sample(-1:1, size = 1, prob = c(0.5*beta,1-beta,0.5*beta))
  final_action <- ((action + delta + 3) %% 4) + 1
  foo <- c(x,y) + unlist(action_deltas[final_action])
  foo <- pmax(c(1,1),pmin(foo,c(H,W)))
  return (foo)
}

q_learning <- function(start_state, epsilon = 0.5, alpha = 0.1, gamma = 0.95,
                       beta = 0){
  current_state <- start_state
  episode_correction <- 0

  repeat{
    # Follow policy, execute action, get reward.
    next_action <- EpsilonGreedyPolicy(current_state[1], current_state[2], epsilon)
    next_state <- transition_model(current_state[1], current_state[2], next_action, beta)
    q_old <- q_table[current_state[1], current_state[2], next_action]
    reward <- reward_map[next_state[1], next_state[2]]

    # Q-table update.
    target <- reward + gamma * max(q_table[next_state[1], next_state[2],])
    q_table[current_state[1], current_state[2], next_action] <<- q_old + alpha * (target - q_old)
    episode_correction <- episode_correction + (target - q_old)

    current_state <- next_state

    if(reward!=0)
      # End episode.
      return (c(reward,episode_correction))
  }
}

MovingAverage <- function(x, n){
  cx <- c(0,cumsum(x))
  rsum <- (cx[(n+1):length(cx)] - cx[1:(length(cx) - n)]) / n
```

```
  return (rsum)
}
H <- 5
W <- 7

reward_map <- matrix(0, nrow = H, ncol = W)
reward_map[3,6] <- 10
reward_map[2:4,3] <- -1

q_table <- array(0,dim = c(H,W,4))

for(i in 1:10000){
  foo <- q_learning(start_state = c(3,1))
  if(any(i==c(10,100,1000,10000)))
    vis_environment(i)
}
H <- 7
W <- 8

reward_map <- matrix(0, nrow = H, ncol = W)
reward_map[1,] <- -1
reward_map[7,] <- -1
reward_map[4,5] <- 5
reward_map[4,8] <- 10

for(eps in c(0.5, 0.1)){
  for(j in c(0.5,0.75,0.95)){
    q_table <- array(0,dim = c(H,W,4))
    reward <- numeric(30000)
    correction <- numeric(30000)

    for(i in 1:30000){
      foo <- q_learning(epsilon = eps, gamma = j, start_state = c(4,1))
      reward[i] <- foo[1]
      correction[i] <- foo[2]
    }

    setting <- paste("epsilon =", eps, ", gamma =", j)
    vis_environment(i, epsilon = eps, gamma = j)

    par(mfrow = c(1,2))
    plot(MovingAverage(reward,100), type = "l", main = paste("Reward,", setting),
         xlab = "Episode", ylab = "Avg reward (last 100)", cex.main = 0.8)
    plot(MovingAverage(correction,100), type = "l", main = paste("Correction,", setting),
         xlab = "Episode", ylab = "Avg correction (last 100)", cex.main = 0.8)
    par(mfrow = c(1,1))
  }
}
H <- 3
W <- 6

reward_map <- matrix(0, nrow = H, ncol = W)
reward_map[1,2:5] <- -1
```

```r
reward_map[1,6] <- 10

for(j in c(0,0.2,0.4,0.66)){
  q_table <- array(0,dim = c(H,W,4))
  for(i in 1:10000)
    foo <- q_learning(gamma = 0.6, beta = j, start_state = c(1,1))
  vis_environment(i, gamma = 0.6, beta = j)
}
```